

Benchmarking SNN pruning algorithms on SATA simulator for efficient energy estimation

Rakshitha Kalkura (CS25E052)

Course Project: CS6886 — IIT Madras

November 11, 2025

Motivation

- SNNs are claimed to be energy-efficient due to spike sparsity.
- But recent studies (“Are SNNs Truly Energy Efficient?”) [6] show this is not always true.
- Pruning reduces operations but its real energy impact must consider memory + compute.
- SNNs are often assumed to be energy-efficient due to sparse activity.
- Recent studies shows training/inference energy can exceed their ANN counterparts.

Problem Statement

Given

ResNet-19 SNN trained on CIFAR-10, pruned using different sparsity methods: STP, uTicket, Unstructured Pruning frameworks

Goals

- How pruning impacts **accuracy, spikes, and energy**
- Estimate inference energy using **SATA** (analytical accelerator model)
- Validate claims using real trends from the literature
- Expose limitations of analytical energy models

Direct Training of SNNs — BPTT

Goal: Train spiking neurons with temporal credit assignment

Key challenge: Spiking function is non-differentiable

$$S(t) = H(V(t) - V_{th}) \Rightarrow \frac{\partial S}{\partial V} = 0$$

Solution: Unroll in time and apply BPTT

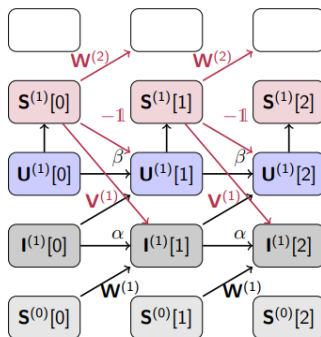
$$\frac{\partial \mathcal{L}}{\partial W} = \sum_{t=1}^T \underbrace{\frac{\partial \mathcal{L}}{\partial S_t}}_{\text{loss signal}} \cdot \underbrace{\frac{\partial S_t}{\partial V_t}}_{0 \rightarrow \text{gradient block}} \cdot \underbrace{\frac{\partial V_t}{\partial W}}_{\text{weight influence}}$$

Implications:

- SNN is treated as an RNN unrolled over T steps
- Temporal dependencies captured, but gradients vanish
- Training fails without gradient approximation

SNN "unrolled": BPTT algorithm

- Input spikes $S(0)$ are fed into the n/w from the bottom and propagate upwards to higher layers.
- The synaptic currents I decay in each time step and are fed into the membrane potentials U .
- Spike trains S are generated via a threshold nonlinearity to the membrane potentials U in each time step.



SNN training with Surrogate gradients

Core Idea: Use a smooth proxy during backprop

$$\frac{\partial S}{\partial V} \approx \sigma'(V)$$

Surrogate function used in our case: $\tanh(.)$ function

Forward vs Backward Path:

- **Forward:** Hard spikes $S = H(V - V_{th})$
- **Backward:** Smooth gradient $\sigma'(V)$ enables learning



Figure: Surrogate gradient approximation

Unstructured Pruning

Core Idea: Remove individual weights with smallest magnitude across the network.

Procedure: Applies unstructured weight pruning (removing individual synaptic connections based on learnable binary masks) and unstructured neuron pruning (removing specific neurons and their connections), both guided by a smooth energy penalty term within the training loss.

Characteristics:

- High sparsity possible (80–95%)
- Irregular memory access (hardware unfriendly)

STP — Spatio-Temporal Pruning

Core Idea: Prunes spiking neural networks (SNNs) by reducing both spatial redundancy (in network weights) and temporal redundancy (in spike timing).

Procedure: Uses Layer-Adaptive Magnitude-based Pruning Score (LAMPS) to balance pruning across layers spatially, and uses KL-divergence of output distributions over time to select an adaptive decision timestep for temporal pruning.

Characteristics:

- More structured sparsity than magnitude pruning
- Smaller accuracy drop at high sparsity

U-Ticket — Lottery Ticket for SNNs

Core Idea: Addresses workload imbalance in extremely sparse SNNs pruned via Lottery Ticket Hypothesis (LTH) methods. Ensures balanced assignment of non-zero weights across processing elements (PEs).

Procedure: Iterative pruning alternating with workload balancing adjustments (recovering/removing weights to make workloads uniform across PEs) before re-initialization. No extra hardware overhead is needed.

Characteristics:

- Preserves accuracy extremely well
- Finds stable subnetwork early in training
- Compatible with standard training pipelines

Accuracy Comparison After Pruning

Method	Accuracy (%)	Sparsity (%)
Baseline (Dense)	91.2	0
Unstructured	88.6	85
UTicket	89.8	80
STP	90.2	60

Comparison of different pruning algorithms

Key Notes:

- Timesteps = 10 across all algorithms
- Architecture: ResNet19
- Dataset: CIFAR10

SATA

SATA (Sparsity-Aware Training Accelerator) is an **analytical energy and latency estimation framework** for SNNs.

Key properties:

- Models energy at **operation level** (MACs, spikes, LIF, memory access)
- Separates energy into: **Compute + Memory + Communication**
- Considers **spike sparsity** through:

$$E \propto S \cdot (\text{SOPs}) + (1 - S) \cdot (\text{idle cost})$$

where S = spike density, SOPs = spike operations

- Provides per-layer energy breakdown: LIF, MAC, SRAM, DRAM
- Supports structured/unstructured sparsity inputs (S , G_f , G_u)

Output: Total inference energy

SATA Simulator – Why?

- Estimates energy of SNN accelerators considering:
 - MAC operations
 - LIF spike updates
 - DRAM, SRAM, SPAD memory access cost
- **Key insight from “Are SNNs Truly Energy Efficient?”:**

Memory movement dominates energy, not compute sparsity → SATA is relevant.

Why not standard FLOP/MAC estimation?

- SNN cost is **not proportional to FLOPs**
- Dominant factor is **spike density + memory movement**
- Sparse models often increase memory irregularity, reducing hardware gains

Implementation : Train + Profile SNN

Step 1: Train ResNet-19 SNN on CIFAR-10

- Direct SNN training via surrogate gradients (ATan surrogate).
- Typical config: $T = 10$ timesteps, batch size 64, CE Loss.

Step 2: Extract layer-wise activity metrics

$$S_l(\text{spike sparsity}) = \frac{\text{non-zero spikes}}{\text{total spikes}}$$

$$G_f(\text{activation gradient sparsity}) = \frac{\text{non-zero activation grads}}{\text{total activations}}$$

$$G_u(\text{Weight Gradient Sparsity}) = \frac{\text{non-zero weight grads}}{\text{total weights}}$$

Implementation: YAML + SATA Inputs

Step 3: Generate SATA network YAML

- Register forward hooks on conv / linear layers to capture output shapes.
- Build YAML fields required by SATA: kernel, stride, in/out channels, output H/W.

Step 4: Save profiling arrays : S_l , G_f , G_u

Implementation: SATA Energy Estimation

Step 5: Run SATA Energy Model

$$E_{total} = (E_c^{fwd} + E_c^{bwd} + E_c^{wup}) \\ + (E_m^{fwd} + E_m^{bwd} + E_m^{wup}) + E_{ctrl}.$$

Outputs:

- Layer-wise compute energy (MAC, Spike-MAC, LIF cost)
- Memory energy (DRAM, SRAM/GLB, SPAD)
- Runtime exported as csv + YAML logs

$$\begin{aligned} E_c^{fwd} &= N_{mac}^{fwd}(sp_S) \times E_{mac}^{fwd} + N_{LIF} \times E_{LIF}, \\ E_c^{bwd} &= N_{mac}^{bwd}(sp_{\nabla U}) \times E_{mac}^{bwd} \\ &\quad + N_{\nabla U}(sp_{\nabla f}) \times E_{\nabla U}, \\ E_c^{wup} &= N_{mac}^{wup}(sp_S) \times E_{mac}^{wup}, \\ E_m^{fwd} &= N_{dram}^{fwd} \times E_{dram} + N_{glb}^{fwd} \times E_{glb} \\ &\quad + N_{spad}^{fwd}(sp_S) \times E_{spad}, \\ E_m^{bwd} &= N_{dram}^{bwd} \times E_{dram} + N_{glb}^{bwd} \times E_{glb} \\ &\quad + N_{spad}^{bwd}(sp_{\nabla f}) \times E_{spad}, \\ E_m^{wup} &= N_{dram}^{wup} \times E_{dram} + N_{glb}^{wup} \times E_{glb} \\ &\quad + N_{spad}^{wup}(sp_S) \times E_{spad}, \end{aligned}$$

SATA energy formulation

Results: SATA Inference Energy & Sparsity

ResNet-19 on CIFAR-10 (SATA Inference Estimate)

Method	Sparsity (%)	Energy (mJ)
Baseline [5]	0	11.0
Unstructured	82.4	6.55
UTicket	88.7	5.7
STP	85.1	3.87

- STP achieves lowest sparsity but also lowest estimated energy
- UTicket provides strong tradeoff without large accuracy loss
- SATA highlights memory vs compute energy contributions per layer

Sanity Check: Do Our SATA Estimates Match Literature?

Reference vs Our SATA Inference Estimates

Work	Sparsity (%)	Reported Energy	Our SATA Est.
Unstructured	80–90%	65–85% energy ↓	40% ↓
UTicket	85–92%	3.5–5.5× energy ↓	1.9 × reduction
STP	90–95%	5–7× energy ↓	2.84 × reduction

Observations

- Differences arise due to:
 - SATA assumes analytical SRAM/DRAM cost model (no hardware schedule effects)
 - Papers report different energy estimation methods

Conclusion: There is a significant difference in SATA estimation and the reported values from papers.

Challenges

- Couldn't reproduce baseline for GradR pruning algorithm
- No estimation on training energy due to limited resources
- Layer studies: Analysis on layer-wise - energy estimation
- Comparison with actual hardware energy
- Temporal study: Behaviour across timesteps

Drawbacks of This Approach

- Energy estimated analytically — no real hardware measurement
- No hardware contention
- Absolute energy estimation is not possible

Conclusion

- Pruning reduces compute, but memory dominates energy
- STP is best for energy, not highest sparsity
- SATA gives relative realistic insight compared to naive FLOP counting

Learnings:

- Research direction, choosing right literature/framework: SATA is great for fast iteration, but it's not full reality.
- Most energy claims in papers look magical until you implement them.
- Sparsity is easy to report, hard to monetize.
- SNNs are “energy efficient” on paper, but real efficiency is a long road.

References

- TOWARDS ENERGY EFFICIENT SPIKING NEURAL NETWORKS: AN UNSTRUCTURED PRUNING FRAMEWORK, Xinyu Shi, ICLR 2024
- Workload-Balanced Pruning for Sparse Spiking Neural Networks, R. Yin, IEEE TRANSACTIONS ON EMERGING TOPICS IN COMPUTATIONAL INTELLIGENCE, VOL. 8, NO. 4, 2024
- Gou S, et. al., (2025) Dynamic spatio-temporal pruning for efficient spiking neural networks. Front. Neurosci.
- SATA: Sparsity-Aware Training Accelerator for Spiking Neural Networks, Ruokai Yin, et. al., - IEEE TRANSACTIONS ON COMPUTER-AIDED DESIGN OF INTEGRATED CIRCUITS AND SYSTEMS, 2022
- Are SNNs Truly Energy-efficient? — A Hardware Perspective, Abhiroop Bhattacharjee, Ruokai Yin , Abhishek Moitra , and Priyadarshini Panda - ICASSP, 2023

Thank You. Questions?